

MEDICUS Application Development

Project Overview and Architecture

Giovanni Montanari

Project Partners: MEDICUS Team & StabLab

September 9, 2026

1 Project Overview and Development Journey

The MEDICUS research team is currently conducting extensive surveys to understand how elderly populations react and adapt to extreme climatic conditions, specifically focusing on air pollution, heat waves, and pollen exposure. To precisely profile this target demographic and uncover critical health insights, the team requires a robust tool to seamlessly explore and visualize both univariate and bivariate relationships within their dataset.

When we first started this project, the goal was simple: build a professional, easy-to-use application to facilitate this exact data exploration. However, we quickly ran into a common problem. Traditional data analysis scripts usually rely on "hardcoded" rules—meaning the code looks for specific column numbers or variable names. The moment a survey changes or a new question is added, those scripts break, making them a nightmare to maintain.

To solve this, we completely changed our approach and built a "configuration-driven" engine. Instead of writing rigid code, we separated the app's logic from the data itself. The star of this new approach is the **Codebook**.

Think of the Codebook as the app's instruction manual. It is a simple CSV file that contains all the rules: the question IDs, the human-readable labels, and the type of data (e.g., whether it is a multiple-choice question or a numeric value).

By building our engine with `shiny`, `data.table`, and `ggplot2`, we achieved several major milestones:

- **The Codebook as the Brain:** The app now reads the Codebook first and automatically figures out how to handle the data. You can swap out the entire dataset and Codebook, and the app will instantly adapt without a single line of R code being rewritten.
- **Smart Data Protection:** We replaced rigid data cleaning steps with a dynamic pipeline. For example, sensitive fields (like data privacy consent) are automatically detected and stripped out by searching the Codebook labels, ensuring they never accidentally show up in the app.
- **Object-Oriented Design:** We built a system that automatically sorts questions into a "Variable Deck" based on the Codebook. The app knows exactly how to visualize a question based on its category (Single Choice, Ordinal, etc.).
- **Zero-Server Deployment:** We deployed the app via WebAssembly (`shinylive`) directly to GitHub Pages. This means the app runs entirely inside your local web browser. No sensitive survey data is ever uploaded to a remote server, which is a massive win for data privacy.

2 App Capabilities and Instructions

When you first open the application, it acts as a blank canvas. It only comes to life once you feed it your data and your Codebook. Figure 1 breaks down how these pieces flow together.

2.1 The Codebook (Configuration)

To use the application, the researcher must provide a "Codebook" formatted as a CSV file. This file acts as the blueprint for the app, dictating how every single column in the raw data should be treated.

As shown in Table 1, the configuration requires a structured layout with specific columns:

item	label	class	scale	breaks
BB01	Alter	single	numerical	59, 69, 74, 79, Inf
BB02	Geschlecht	single	categorical	
CC02	Beschwerden	single	howmuch	
DD01	Infos woher	multiple		

Table 1: Example extract from the project’s CSV Codebook.

- **item:** The unique identifier matching the dataset column (e.g., BB01).
- **label:** The human-readable question text displayed in the user interface (e.g., "Alter" or "Geschlecht").
- **class:** The broad format of the question (e.g., **single**, **multiple**).
- **scale:** The specific measurement level or named scale to apply (e.g., **numerical**, **categorical**, **howmuch**, **freq**).
- **breaks:** Optional boundaries used to categorize numerical data into distinct groups (e.g., defining age brackets).

2.2 How to Use the App

We designed the interface to be incredibly intuitive, especially for non-technical researchers. Here is how you use it:

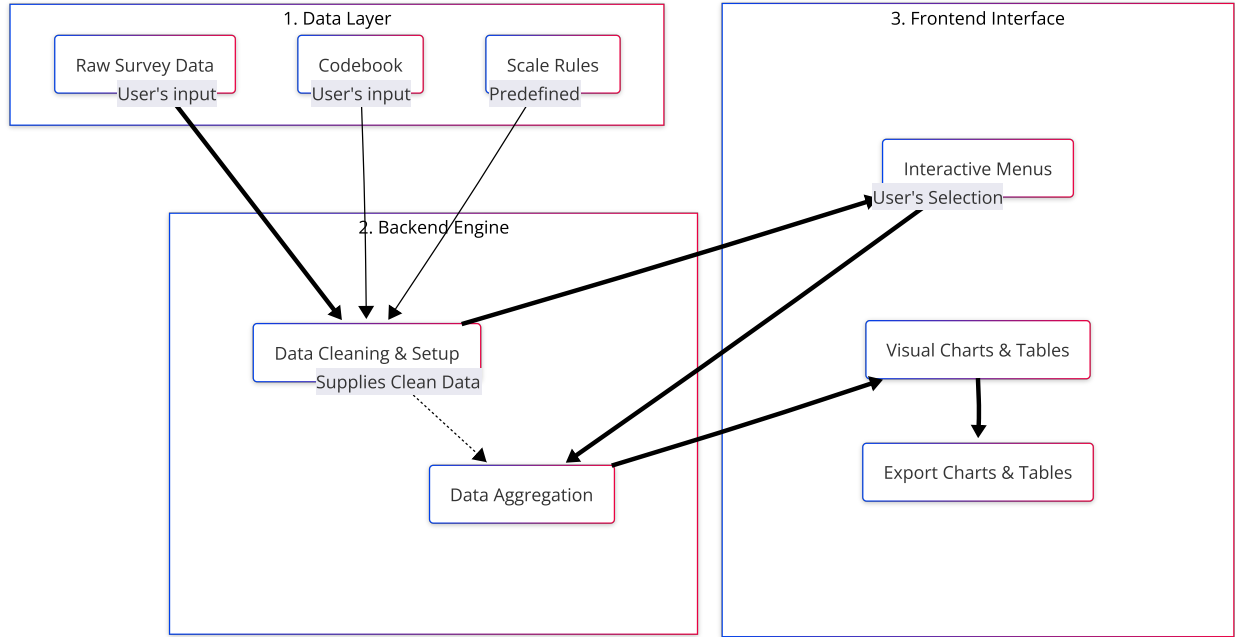


Figure 1: Three-Tier Architecture of the Data Pipeline

1. **Upload Your Files:** On the left side of the screen, you will see upload buttons. Drop in your raw survey dataset (as an Excel or CSV file) and your Codebook (as a CSV).
2. **Let the Engine Run:** Click "Daten verarbeiten" (Process Data). In the background, the app cleans the data, pairs it with your Codebook rules, and sets up the dashboard.
3. **Explore the Data:** Dropdown menus will automatically appear. You can pick a main question to plot on the X-axis, and if you want to dig deeper, pick a second question to group the data by color.
4. **Customize and Export:** You can easily toggle between seeing absolute numbers or percentage shares. If you see a table you like, just click the export button to download it directly to your computer as a CSV for your reports.

3 Possible future improvements

While we have built a highly flexible foundation, the MEDICUS team has already brought forward some excellent ideas for making the app even more powerful. Should they decide to invest additional project hours with StabLab, these concepts could be explored in a future development cycle:

- **Generalized Scales Manager:** Currently, the predefined ordinal scales used in the application were saved based specifically on the very first survey we received. For example, a question asking "How often do you experience..." is currently tied to a fixed scale ranging from "never" to "always." Together with the MEDICUS team we talked about building an administrative dashboard. If implemented, this would allow researchers to create, save, and manage their own standard scales globally, making the Codebook even simpler to write for future surveys.
- **Item Indexing and Scoring:** Another idea proposed by the MEDICUS team is the ability to index and score items. If explored, we could add backend logic that groups related survey questions together. This would allow the application to automatically calculate overall composite scores, averages, or sum indices—a feature that would be incredibly useful for evaluating validated psychological or medical questionnaires.